

Faculty of Computers and Information Systems

Information Retrieval
Practical Laboratory

Week 7: Term Frequency Document Frequency and TF-IDF

Replace binary occurrence values with numerical term weights. Calculate a small example by hand, then compute a complete TF-IDF matrix for the Week 6 collection.

Duration: approximately 2 practical hours. Suggested pacing: 15 minutes concept review, 25 demonstration, 50 guided implementation, 20 selected exercises, and 10 review. Finish remaining exercises after the session.

1. Lab Objectives

- Count term frequencies within individual documents.
- Count document frequencies across the collection.
- Calculate natural-log inverse document frequency.
- Construct and interpret TF-IDF vectors without external packages.

2. Required Background

Prerequisite: basic Python variables, conditions, loops, lists, dictionaries, sets, and functions. Complete the earlier labs before starting this week.

$TF(t,d)$ is the number of occurrences of term t in document d after preprocessing. $DF(t)$ counts documents containing t , not token occurrences. N is the number of loaded documents, including a document whose preprocessing yields no tokens.

Use $IDF(t) = \log(N / DF(t))$, with `math.log` providing the natural logarithm. $TF-IDF(t,d) = TF(t,d) \times IDF(t)$. This lab uses raw counts, no smoothing, and no separate vector normalization. A term in every document has IDF zero; a rarer term receives a larger IDF.

Only vocabulary terms have an IDF entry, so each computed DF is at least one. An unknown term has no vocabulary coordinate and no $\log(N/0)$ is calculated. An empty collection returns empty structures. A document with no retained tokens has an all-zero vector.

All document vectors share one sorted vocabulary. The i -th coordinate always represents `vocabulary[i]`. Keep this coordinate order with the vectors so dot products in Week 8 compare matching terms.

$$TF(t,d) = \text{number of occurrences of } t \text{ in } d$$

$$DF(t) = \text{number of documents containing } t$$

$$IDF(t) = \ln(N / DF(t))$$

$$TF-IDF(t,d) = TF(t,d) \times IDF(t)$$

Course material: Extension of the supplied document loader and tokenizer. The weighting formulas follow the requested syllabus.

3. Python Concepts Used

- **Counting dictionary:** Use `counts.get(term, 0) + 1` for raw term frequency.
- **Unique dictionary keys:** Count each document at most once for each DF entry.
- **math.log:** Compute natural-log inverse document frequency.
- **Floating-point arithmetic:** Retain full precision internally and round only for display.
- **List vectors:** Build one value per term in the same vocabulary order.

4. Worked Example

Hand calculation first: let $d1 = \text{"data data python"}$, $d2 = \text{"data java"}$, and $d3 = \text{"java"}$. With $N = 3$, calculate TF, DF, IDF, and every nonzero TF-IDF entry before running the check below.

```
import math
documents = {"d1": ["data", "data", "python"],
            "d2": ["data", "java"], "d3": ["java"]}
for term in ["data", "java", "python"]:
    df = sum(term in tokens for tokens in documents.values())
    idf = math.log(len(documents) / df)
    weights = [tokens.count(term) * idf
               for tokens in documents.values()]
    print(term, df, round(idf, 4),
          [round(value, 4) for value in weights])
```

$DF(\text{data}) = DF(\text{java}) = 2$ and $DF(\text{python}) = 1$. Thus $IDF(\text{data}) = IDF(\text{java}) = \ln(3/2) \approx 0.4055$ and $IDF(\text{python}) = \ln(3) \approx 1.0986$. In $d1$, the data weight is $2 \times 0.405465... \approx 0.8109$. Do not round IDF before multiplying in the program.

Term	DF	IDF	d1 TF-IDF	d2 TF-IDF	d3 TF-IDF
data	2	0.4055	0.8109	0.4055	0.0000
java	2	0.4055	0.0000	0.4055	0.4055
python	1	1.0986	1.0986	0.0000	0.0000

5. Step-by-Step Practical Task

Step 1: Complete the three-document hand calculation above before opening the solution.

Step 2: Load exactly the ten documents from Week 6 and tokenize each one.

Step 3: Count TF values using a dictionary, preserving repeated tokens.

Step 4: For each document, increment DF once for each distinct term in that document.

Step 5: Calculate IDF with `math.log` and construct vectors using one sorted vocabulary.

Step 6: Print the full matrix. It appears in blocks of five document columns so it remains readable. Compare selected entries with independent hand calculations.

Execution: open a terminal in this week folder and run `python lab07_solution.py`. On Windows, `py -3` can replace `python`. All required files are included; no internet or third-party Python packages are needed.

6. Complete Python Program

The listing below matches `lab07_solution.py` exactly. Keep indentation when copying. The exercise starter is separate and intentionally unfinished.

lab07_solution.py

```
import os
import re

STOP_WORDS = {
    "a", "an", "the", "is", "in", "for", "and", "are", "with",
    "like", "of", "to", "it", "be", "used", "uses", "that", "this",
    "was", "were", "by", "from", "on", "at", "or",
}

SAMPLE_DOCUMENTS = {
    "doc1": "Python is great for data science and machine learning",
    "doc2": "Java is used for enterprise applications and web development",
    "doc3": "Machine learning uses neural networks and decision trees",
```

```

    "doc4": "Python supports data analysis with libraries like Pandas",
    "doc5": "Java and C++ are used for game development and graphics",
}
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

def load_documents(folder_path):
    """Load UTF-8 .txt files; use five samples if missing or empty."""
    documents = {}
    if os.path.isdir(folder_path):
        for filename in sorted(os.listdir(folder_path)):
            filepath = os.path.join(folder_path, filename)
            if filename.endswith(".txt") and os.path.isfile(filepath):
                with open(filepath, "r", encoding="utf-8") as file:
                    documents[filename[:-4]] = file.read()
    if not documents:
        return SAMPLE_DOCUMENTS.copy()
    return documents

def tokenize(text):
    """Use the same ASCII-letter tokens as the supplied course files."""
    words = re.findall(r"[a-z]+", text.lower())
    return [word for word in words if word not in STOP_WORDS]

import math

def term_frequency(tokens):
    counts = {}
    for term in tokens:
        counts[term] = counts.get(term, 0) + 1
    return counts

def build_tfidf(documents):
    """Raw TF and natural-log IDF; no smoothing or extra normalization."""
    counts = {}
    document_frequency = {}
    for name, text in documents.items():
        counts[name] = term_frequency(tokenize(text))
        for term in counts[name]:
            document_frequency[term] = document_frequency.get(term, 0) + 1
    vocabulary = sorted(document_frequency)
    idf = {}
    total = len(documents)
    for term in vocabulary:
        idf[term] = math.log(total / document_frequency[term])
    vectors = {}
    for name in sorted(documents):
        vectors[name] = [
            counts[name].get(term, 0) * idf[term] for term in vocabulary
        ]
    return vocabulary, counts, document_frequency, idf, vectors

def main():
    documents = load_documents(os.path.join(BASE_DIR, "docs"))
    vocabulary, counts, df, idf, vectors = build_tfidf(documents)
    names = sorted(documents)
    print("Documents:", names)
    print("Term      DF      IDF      TF in doc9")
    for term in ["python", "data", "retrieval"]:
        print(f"{term:12} {df.get(term, 0):2} {idf.get(term, 0):8.4f}",
              counts.get("doc9", {}).get(term, 0))
    # Five columns per block keep the full matrix readable on paper.
    for start in range(0, len(names), 5):
        block = names[start:start + 5]
        print("\nTF-IDF matrix:", " ".join(block))
        for i, term in enumerate(vocabulary):
            values = " ".join(f"{vectors[name][i]:7.3f}" for name in block)
            print(f"{term:12} {values}")

```

```
if __name__ == "__main__":
    main()
```

7. Expected Output

Representative output from the shipped collection and the default command. Calculated values come from a tested program run. Long output lines wrap here for printing; expected_output.txt contains the full terminal output. Selected rows are shown for large matrices.

Documents: ['doc1', 'doc10', 'doc2', 'doc3', 'doc4', 'doc5', 'doc6', 'doc7', 'doc8', 'doc9']

Term	DF	IDF	TF in doc9
python	2	1.6094	0
data	5	0.6931	3
retrieval	2	1.6094	0

TF-IDF matrix:	doc1	doc10	doc2	doc3	doc4
data	0.693	0.000	0.000	0.000	0.693
python	1.609	0.000	0.000	0.000	1.609
retrieval	0.000	1.609	0.000	0.000	0.000

TF-IDF matrix:	doc5	doc6	doc7	doc8	doc9
data	0.000	1.386	0.693	0.000	2.079
python	0.000	0.000	0.000	0.000	0.000
retrieval	0.000	0.000	0.000	1.609	0.000

These results use all ten shipped documents. Editing the collection changes the index and may change weights, ranks, and evaluation results.

8. Laboratory Exercises

1. Hand calculation: add $d4 = \text{"data"}$ to the worked example and recalculate $DF(\text{data})$, $IDF(\text{data})$, and the $d1$ data weight.
2. Implementation: finish `tfidf_weight` in the starter for known terms with $1 \leq DF \leq N$.
3. Debugging: explain why incrementing DF for every token occurrence can produce a negative IDF .
4. Interpretation: add the same retained term to every document and explain its resulting IDF .
5. Implementation: compare DF values with posting-list lengths from Week 6.
6. Extension: repeat a term in one document and describe which TF , DF , IDF , and $TF \cdot IDF$ values change.

9. Student Tasks

- Your complete hand calculation and the completed weighting function.
- The generated matrix and independent checks for at least three entries.
- An explanation of the effects of repetition and collection-wide common terms.

Submit your own implementation and evidence. Do not submit an untouched starter as a completed solution. Label all outputs with the query or input that produced them.

10. Review Questions

1. How do TF and DF differ?
2. Which logarithm does `math.log` use?
3. Why is the IDF of a universal term zero?
4. Why is there no IDF coordinate for an unknown term?
5. Why must all vectors share a vocabulary order?

11. Challenge Exercise

Optional: Implement log-scaled TF as a separate experiment, leaving the required raw- TF baseline unchanged. Compare the resulting weights for a term repeated once, twice, and ten times.